

MODUL PRAKTIKUM STRUKTUR DATA

Modul 4



Disusun Oleh:

Agus Nugraha, ST, M.Kom | Fitri Fatimah | Kanaya Dzikra

**PROGRAM STUDI SAINS DATA
FAKULTAS SAINS DAN TEKNOLOGI**

PERTEMUAN 4

ANTRIAN (QUEUE) DAN DEQUE

1. DESKRIPSI MODUL

Modul ini membahas struktur data linear Queue (Antrian) dan Deque (Double Ended Queue). Queue merupakan struktur data yang bekerja berdasarkan prinsip First In First Out (FIFO), yaitu data yang pertama masuk akan menjadi data pertama yang keluar.

Selain Queue, mahasiswa juga akan mempelajari Deque yang merupakan pengembangan Queue dengan kemampuan melakukan penambahan dan penghapusan data dari kedua sisi, baik depan maupun belakang.

Konsep Queue dan Deque banyak digunakan dalam sistem komputer modern seperti antrian printer, sistem tiket online, call center, penjadwalan proses sistem operasi, sistem rumah sakit, dan aplikasi navigasi.

Melalui praktikum ini mahasiswa akan memahami konsep FIFO, operasi dasar Queue dan Deque, implementasi menggunakan Python, serta penerapannya dalam berbagai studi kasus dunia nyata.

2. TUJUAN PRAKTIKUM

Setelah mengikuti praktikum ini mahasiswa diharapkan mampu:

- Memahami konsep Queue dan Deque.
- Memahami prinsip FIFO.
- Mengimplementasikan operasi enqueue dan dequeue.
- Mengimplementasikan operasi pada Deque.
- Menganalisis perubahan isi Queue setelah setiap operasi.
- Menggunakan Queue dan Deque untuk menyelesaikan studi kasus sederhana.

3. CAPAIAN PEMBELAJARAN MODUL (CPM)

Mahasiswa mampu:

- Menjelaskan konsep Queue dan Deque.
- Mengimplementasikan Queue menggunakan Python.
- Mengimplementasikan Deque menggunakan Python.
- Menganalisis proses masuk dan keluarnya data.
- Memilih struktur data yang sesuai untuk suatu permasalahan.

4. DASAR TEORI

4.1 Pengertian Queue

Queue adalah struktur data linear yang menggunakan prinsip: **FIFO (First In First Out)**. Artinya data yang pertama masuk akan menjadi data pertama yang keluar.

4.2 Analogi Queue

Antrian Bank

Misalkan terdapat antrian:

Front → [Andi, Budi, Citra] ← Rear

Andi datang lebih dahulu sehingga akan dilayani terlebih dahulu.

Jika dilakukan dequeue:

Andi keluar

Maka tersisa:

[Budi, Citra]

Antrian Printer

Dokumen yang pertama dikirim ke printer akan dicetak terlebih dahulu.

Call Center

Penelepon yang masuk lebih dahulu akan mendapatkan layanan lebih dahulu.

4.3 Struktur Queue

Queue memiliki dua posisi penting:

- **Front** = posisi data yang akan keluar terlebih dahulu.
- **Rear** = posisi tempat data baru masuk.

Contoh:

Front

↓

[A] [B] [C]

↑

Rear

4.4 Operasi Dasar Queue

- **Enqueue** = menambahkan data ke belakang antrian.
- **Dequeue** = menghapus data dari depan antrian.
- **Front / Peek** = melihat data paling depan tanpa menghapusnya.
- **IsEmpty** = mengecek apakah Queue kosong.

4.5 Kompleksitas Queue

Operasi	Kompleksitas
Enqueue	O(1)
Dequeue	O(1)
Peek	O(1)

Queue termasuk struktur data yang efisien.

4.6 Pengertian Deque

Deque (Double Ended Queue) adalah struktur data yang memungkinkan:

- Penambahan data dari depan
- Penambahan data dari belakang
- Penghapusan data dari depan
- Penghapusan data dari belakang

4.7 Operasi Deque

- **Add Front** = menambah data dari depan.
- **Add Rear** = menambah data dari belakang.
- **Remove Front** = menghapus data dari depan.
- **Remove Rear** = menghapus data dari belakang.

4.8 Perbedaan Queue dan Deque

Queue	Deque
FIFO	Lebih fleksibel
Tambah dari belakang	Tambah dari dua sisi
Hapus dari depan	Hapus dari dua sisi
Lebih sederhana	Lebih dinamis

5. ALAT DAN PERANGKAT

- Laptop / Komputer
- Python
- Google Colab
- Browser
- Modul Praktikum

6. PROSEDUR PRAKTIKUM

Praktikum 1 – Membuat Queue Sederhana

```
queue = []  
  
print(queue)
```

Analisis = membuat Queue kosong menggunakan list Python.

Praktikum 2 – Operasi Enqueue

```
queue = []  
  
queue.append("Andi")  
  
queue.append("Budi")
```

```
queue.append("Citra")  
print(queue)
```

Output = ['Andi', 'Budi', 'Citra']

Praktikum 3 – Operasi Dequeue

```
queue = ["Andi", "Budi", "Citra"]  
keluar = queue.pop(0)  
print("Dilayani:", keluar)  
print(queue)
```

Output

Dilayani: Andi

['Budi', 'Citra']

Analisis = Data pertama yang masuk keluar terlebih dahulu.

Praktikum 4 – Operasi Peek

```
queue = ["Andi", "Budi", "Citra"]  
print(queue[0])
```

Output = Andi

Praktikum 5 – IsEmpty

```
queue = []  
  
if len(queue) == 0:  
    print("Queue Kosong")
```

Praktikum 6 – Simulasi Antrian Klinik

```
queue = []  
  
queue.append("Andi")  
queue.append("Budi")
```

```
queue.append("Citra")  
  
print("Sebelum:", queue)
```

```
queue.pop(0)  
  
print("Sesudah:", queue)
```

Praktikum 7 - Simulasi Antrian Printer

```
queue = []  
  
queue.append("Dokumen A")  
queue.append("Dokumen B")  
queue.append("Dokumen C")
```

```
print(queue)
```

```
queue.pop(0)  
  
print(queue)
```

Praktikum 8 - Membuat Deque

```
from collections import deque  
  
dq = deque()  
  
print(dq)
```

Praktikum 9 - Add Rear

```
from collections import deque  
  
dq = deque()  
  
dq.append("A")  
  
dq.append("B")
```

```
print(dq)
```

Praktikum 10 – Add Front

```
from collections import deque
```

```
dq = deque()
```

```
dq.appendleft("VIP")
```

```
dq.append("Regular")
```

```
print(dq)
```

Output = deque(['VIP', 'Regular'])

Praktikum 11 – Remove Front

```
dq.popleft()
```

Praktikum 12 – Remove Rear

```
dq.pop()
```

Praktikum 13 – Simulasi Rumah Sakit

```
from collections import deque
```

```
antrian = deque()
```

```
antrian.append("Pasien A")
```

```
antrian.append("Pasien B")
```

```
antrian.appendleft("Darurat")
```

```
print(antrian)
```

Analisis = Pasien darurat ditempatkan di depan antrian sehingga dilayani lebih dahulu.

7. STUDI KASUS

Studi Kasus 1

Antrian pasien di klinik kesehatan.

Studi Kasus 2

Antrian dokumen printer.

Studi Kasus 3

Sistem tiket konser online.

Studi Kasus 4

Call center pelanggan.

Studi Kasus 5

Rumah sakit dengan pasien prioritas.

Studi Kasus 6

Sistem parkir VIP menggunakan Deque.

8. TUGAS PRAKTIKUM

Tugas 1

Buat Queue dan masukkan data: 10, 20, 30, 40

Tugas 2

Lakukan dua kali Dequeue.

Tugas 3

Tampilkan isi Queue setelah Dequeue.

Tugas 4

Buat simulasi antrian bank.

Tugas 5

Buat simulasi antrian printer.

Tugas 6

Buat Deque dan tambahkan data dari depan dan belakang.

Tugas 7

Buat simulasi pasien prioritas rumah sakit.

Tugas 8

Analisis perbedaan Queue dan Deque.

Tugas 9

Tentukan struktur data yang tepat untuk:

- Antrian kasir
- Sistem VIP
- Call center

9. DISKUSI

1. Apa perbedaan FIFO dan LIFO?
2. Mengapa Queue cocok untuk sistem antrian?
3. Apa fungsi Front dan Rear?
4. Mengapa Deque lebih fleksibel dibanding Queue?
5. Kapan Queue digunakan?
6. Kapan Deque digunakan?
7. Mengapa pasien prioritas lebih cocok menggunakan Deque?

10. FORMAT LAPORAN

1. Judul Praktikum
2. Tujuan
3. Dasar Teori
4. Langkah Praktikum
5. Hasil dan Screenshot
6. Analisis
7. Kesimpulan

MODUL PRAKTIKUM STRUKTUR DATA

Modul 5



Disusun Oleh:

Agus Nugraha, ST, M.Kom | Fitri Fatimah | Kanaya Dzikra

**PROGRAM STUDI SAINS DATA
FAKULTAS SAINS DAN TEKNOLOGI**

PERTEMUAN 5

DAFTAR TERTAUT (LINKED LIST)

1. DESKRIPSI MODUL

Modul ini membahas struktur data dinamis Linked List (Daftar Tertaut) yang terdiri dari sekumpulan node yang saling terhubung. Berbeda dengan array yang menyimpan data secara berurutan dalam memori, Linked List menyimpan data dalam node-node yang terhubung melalui referensi (link).

Linked List memberikan fleksibilitas dalam penambahan dan penghapusan data karena tidak memerlukan pergeseran elemen seperti pada array. Oleh karena itu, Linked List sering digunakan pada sistem yang membutuhkan perubahan data secara dinamis.

Pada praktikum ini mahasiswa akan mempelajari konsep node, pointer (reference), struktur linked list, operasi dasar linked list, serta perbandingannya dengan array.

2. TUJUAN PRAKTIKUM

Setelah mengikuti praktikum ini mahasiswa diharapkan mampu:

- Memahami konsep Linked List.
- Memahami struktur node dan reference.
- Mengimplementasikan Single Linked List.
- Melakukan operasi insert, delete, dan traversal.
- Menganalisis perbedaan Array dan Linked List.
- Memahami kelebihan dan kekurangan struktur data dinamis.

3. CAPAIAN PEMBELAJARAN MODUL (CPM)

Mahasiswa mampu:

- Menjelaskan konsep Linked List.
- Membuat node sederhana menggunakan Python.
- Mengimplementasikan Single Linked List.
- Melakukan traversal pada Linked List.
- Melakukan operasi insert dan delete.

- Membandingkan Linked List dan Array.

4. DASAR TEORI

4.1 Pengertian Linked List

Linked List adalah struktur data yang terdiri dari node-node yang saling terhubung.

Setiap node memiliki:

- Data
- Link (pointer/reference)

4.2 Struktur Node

Ilustrasi:

[10 | •] ----> [20 | •] ----> [30 | None]

Penjelasan:

- Node pertama menyimpan 10
- Node kedua menyimpan 20
- Node ketiga menyimpan 30
- Node terakhir menunjuk ke None

4.3 Komponen Linked List

- **Data** = Nilai yang disimpan dalam node.
- **Next** = Alamat atau referensi menuju node berikutnya.

4.4 Mengapa Menggunakan Linked List?

Pada Array: [10][20][30][40]

Jika ingin menyisipkan data di tengah: [10][20][25][30][40]

Semua data setelah 20 harus digeser.

Pada Linked List: 10 -> 20 -> 30 -> 40

Cukup mengubah hubungan antar node: 10 -> 20 -> 25 -> 30 -> 40

Tanpa menggeser data lain.

4.5 Single Linked List

Single Linked List adalah linked list yang setiap node hanya memiliki satu pointer menuju node berikutnya.

Ilustrasi: A -> B -> C -> D -> None

Arah pergerakan hanya maju.

4.6 Operasi Dasar Linked List

- **Insert** = Menambahkan node baru.
- **Delete** = Menghapus node tertentu.
- **Traversal** = Menelusuri seluruh node dari awal hingga akhir.
- **Search** = Mencari node tertentu.

4.7 Kompleksitas Operasi

Operasi	Kompleksitas
Traversal	$O(n)$
Search	$O(n)$
Insert Awal	$O(1)$
Delete Awal	$O(1)$

4.8 Perbandingan Array dan Linked List

Array	Linked List
Ukuran tetap	Dinamis
Akses cepat	Traversal berurutan
Insert lambat	Insert lebih fleksibel
Delete lambat	Delete lebih fleksibel

5. ALAT DAN PERANGKAT

- Laptop / Komputer
- Python
- Google Colab
- Browser

- Modul Praktikum

6. PROSEDUR PRAKTIKUM

Praktikum 1 – Membuat Node

```
class Node:

    def __init__(self, data):

        self.data = data

        self.next = None
```

```
node1 = Node(10)

print(node1.data)
```

Analisis = Node memiliki data dan next.

Praktikum 2 – Membuat Dua Node

```
class Node:

    def __init__(self, data):

        self.data = data

        self.next = None
```

```
node1 = Node(10)

node2 = Node(20)

node1.next = node2

print(node1.next.data)
```

Output = 20

Praktikum 3 – Membuat Linked List Sederhana

```
class Node:

    def __init__(self, data):

        self.data = data

        self.next = None
```

```
A = Node("A")
```

```
B = Node("B")
```

```
C = Node("C")
```

```
A.next = B
```

```
B.next = C
```

Praktikum 4 – Traversal Linked List

```
current = A

while current:

    print(current.data)

    current = current.next
```

Output

A

B

C

Praktikum 5 – Insert di Awal

```
D = Node("D")
```

```
D.next = A
```

```
A = D
```


Hasil

D -> A -> B -> C

Praktikum 6 – Insert di Akhir

```
C.next = Node("E")
```

Hasil

A -> B -> C -> E

Praktikum 7 – Delete Node

```
A.next = C
```

Hasil

A -> C

Node B tidak lagi terhubung.

Praktikum 8 – Search Node

```
target = "B"
```

```
current = A
```

```
while current:
    if current.data == target:
        print("Ditemukan")
        break
    current = current.next
```

Praktikum 9 – Simulasi Daftar Mahasiswa

```
A = Node("Andi")
```

```
B = Node("Budi")
```

```
C = Node("Citra")
```

A.next = B

B.next = C

Traversal seluruh mahasiswa.

Praktikum 10 – Analisis Linked List

Bandingkan:

Array

data = [10,20,30,40]

Linked List

10 -> 20 -> 30 -> 40

Diskusikan:

- kemudahan insert
- kemudahan delete
- penggunaan memori

7. STUDI KASUS

Studi Kasus 1

Daftar mahasiswa dalam sistem akademik.

Studi Kasus 2

Playlist musik.

Studi Kasus 3

Riwayat transaksi.

Studi Kasus 4

Daftar antrean pelanggan.

Studi Kasus 5

Data produk pada toko online.

8. TUGAS PRAKTIKUM

Tugas 1

Buat 3 node berisi angka:

10 -> 20 -> 30

Tugas 2

Lakukan traversal.

Tugas 3

Tambahkan node baru di awal.

Tugas 4

Tambahkan node baru di akhir.

Tugas 5

Hapus salah satu node.

Tugas 6

Buat program pencarian data.

Tugas 7

Bandingkan Linked List dan Array.

Tugas 8

Jelaskan kelebihan dan kekurangan Linked List.

Tugas 9

Buat simulasi daftar mahasiswa menggunakan Linked List.

9. DISKUSI

1. Apa yang dimaksud dengan node?
2. Apa fungsi next?
3. Mengapa Linked List disebut struktur data dinamis?
4. Apa kelebihan Linked List dibanding Array?

5. Apa kekurangan Linked List?
6. Mengapa traversal diperlukan?
7. Kapan Linked List lebih cocok digunakan daripada Array?

10. FORMAT LAPORAN

1. Judul Praktikum
2. Tujuan
3. Dasar Teori
4. Langkah Praktikum
5. Hasil dan Screenshot
6. Analisis
7. Kesimpulan

MODUL PRAKTIKUM STRUKTUR DATA

Modul 6



Disusun Oleh:

Agus Nugraha, ST, M.Kom | Fitri Fatimah | Kanaya Dzikra

**PROGRAM STUDI SAINS DATA
FAKULTAS SAINS DAN TEKNOLOGI**

PERTEMUAN 6

POHON (TREE) DAN POHON BINER (BINARY TREE)

1. DESKRIPSI MODUL

Modul ini membahas struktur data non-linear Tree (Pohon) dan Binary Tree (Pohon Biner). Tree digunakan untuk merepresentasikan hubungan hierarki antar data, di mana setiap data disusun dalam bentuk node-node yang saling terhubung.

Berbeda dengan struktur data linear seperti array, stack, queue, dan linked list, tree memungkinkan data memiliki hubungan bercabang sehingga lebih sesuai untuk merepresentasikan struktur organisasi, sistem folder komputer, silsilah keluarga, dan berbagai struktur hierarkis lainnya.

Pada praktikum ini mahasiswa akan mempelajari konsep dasar tree, bagian-bagian tree, binary tree, serta teknik traversal untuk menelusuri seluruh node dalam tree.

2. TUJUAN PRAKTIKUM

Setelah mengikuti praktikum ini mahasiswa diharapkan mampu:

- Memahami konsep Tree dan Binary Tree.
- Mengidentifikasi bagian-bagian Tree.
- Memahami hubungan parent dan child.
- Mengimplementasikan Binary Tree sederhana menggunakan Python.
- Melakukan traversal tree.
- Membedakan preorder, inorder, dan postorder traversal.

3. CAPAIAN PEMBELAJARAN MODUL (CPM)

Mahasiswa mampu:

- Menjelaskan konsep Tree dan Binary Tree.
- Mengidentifikasi root, parent, child, leaf, dan subtree.
- Membuat Binary Tree sederhana.
- Mengimplementasikan traversal pada Tree.
- Menganalisis struktur hierarki menggunakan Tree.

4. DASAR TEORI

4.1 Pengertian Tree

Tree adalah struktur data non-linear yang terdiri dari sekumpulan node yang terhubung secara hierarkis.

Tree digunakan untuk merepresentasikan hubungan bertingkat.

Contoh:

Direktur

|— Manager A

| |— Staff 1

| |— Staff 2

|— Manager B

|— Staff 3

|— Staff 4

4.2 Terminologi Tree

Root adalah node paling atas dalam Tree.

Contoh:

A

/ \

B C

A adalah root.

Parent adalah node yang memiliki anak.

Contoh:

A

|— B

|— C

A adalah parent dari B dan C.

Child adalah node yang berada di bawah parent.

B dan C adalah child dari A.

Leaf adalah node yang tidak memiliki child.

Contoh:



Leaf:

B, D, E

Subtree adalah bagian kecil dari Tree yang juga membentuk Tree.

Contoh:



merupakan subtree.

4.3 Mengapa Menggunakan Tree?

Tree cocok digunakan untuk:

- Struktur folder komputer
- Struktur organisasi
- Silsilah keluarga
- Kategori produk
- Struktur website

4.4 Binary Tree

Binary Tree adalah Tree yang setiap node memiliki maksimal dua child.

Yaitu:

- Left Child
- Right Child

Contoh:



4.5 Struktur Node Binary Tree

Setiap node memiliki: Data, Left dan Right

Ilustrasi: [Data | Left | Right]

4.6 Traversal Tree

Traversal adalah proses mengunjungi seluruh node dalam tree.

Preorder Traversal

Urutan:

Root → Left → Right

Contoh:



Hasil:

A B C

Inorder Traversal

Urutan:

Left → Root → Right

Contoh:

A

/ \

B C

Hasil:

B A C

Postorder Traversal

Urutan:

Left → Right → Root

Contoh:

A

/ \

B C

Hasil: B C A

4.7 Perbandingan Struktur Data

Struktur	Bentuk
Array	Linear
Stack	Linear
Queue	Linear
Linked List	Linear
Tree	Non-linear

5. ALAT DAN PERANGKAT

- Laptop / Komputer
- Python
- Google Colab
- Browser
- Modul Praktikum

6. PROSEDUR PRAKTIKUM

Praktikum 1 – Membuat Node Tree

```
class Node:

    def __init__(self, data):

        self.data = data

        self.left = None

        self.right = None
```

```
root = Node("A")
```

```
print(root.data)
```

Analisis = Membuat satu node sebagai root.

Praktikum 2 – Membuat Binary Tree Sederhana

```
class Node:

    def __init__(self, data):

        self.data = data

        self.left = None

        self.right = None
```

```
root = Node("A")
```

```
root.left = Node("B")
root.right = Node("C")
```

Struktur:

```
  A
 /\
B  C
```

Praktikum 3 – Menambahkan Child

```
root.left.left = Node("D")
root.left.right = Node("E")
```

Struktur:

```
  A
 /\
B  C
 /\
D  E
```

Praktikum 4 – Traversal Preorder

```
def preorder(node):
    if node:
        print(node.data, end=" ")
        preorder(node.left)
        preorder(node.right)

preorder(root)
```

Output: A B D E C

Praktikum 5 – Traversal Inorder

```
def inorder(node):  
    if node:  
        inorder(node.left)  
        print(node.data, end=" ")  
        inorder(node.right)  
  
inorder(root)
```

Output: D B E A C

Praktikum 6 – Traversal Postorder

```
def postorder(node):  
    if node:  
        postorder(node.left)  
        postorder(node.right)  
        print(node.data, end=" ")  
  
postorder(root)
```

Output: D E B C A

Praktikum 7 – Menghitung Jumlah Node

```
def count_nodes(node):  
    if node is None:  
        return 0  
  
    return 1 + count_nodes(node.left) + count_nodes(node.right)  
  
print(count_nodes(root))
```

Praktikum 8 – Menghitung Leaf Node

```
def count_leaf(node):  
    if node is None:  
        return 0  
  
    if node.left is None and node.right is None:  
        return 1  
  
    return count_leaf(node.left) + count_leaf(node.right)  
  
print(count_leaf(root))
```

Praktikum 9 – Menentukan Tinggi Tree

```
def height(node):  
    if node is None:  
        return 0  
  
    return max(height(node.left), height(node.right)) + 1  
  
print(height(root))
```

Praktikum 10 – Visualisasi Struktur Organisasi

Buat Tree yang menggambarkan:

Direktur

├── Manager A

| ├── Staff 1

| └── Staff 2

└── Manager B

Kemudian lakukan traversal.

7. STUDI KASUS

Studi Kasus 1

Struktur organisasi perusahaan.

Studi Kasus 2

Struktur folder komputer.

Studi Kasus 3

Silsilah keluarga.

Studi Kasus 4

Kategori produk toko online.

Studi Kasus 5

Menu aplikasi bertingkat.

8. TUGAS PRAKTIKUM

Tugas 1

Buat Binary Tree berikut:

A

/ \

B C

/ \

D E

Tugas 2

Lakukan traversal preorder.

Tugas 3

Lakukan traversal inorder.

Tugas 4

Lakukan traversal postorder.

Tugas 5

Hitung jumlah node.

Tugas 6

Hitung jumlah leaf node.

Tugas 7

Hitung tinggi tree.

Tugas 8

Buat Tree yang menggambarkan struktur organisasi kelas.

Tugas 9

Jelaskan perbedaan Linked List dan Tree.

9. DISKUSI

1. Apa yang dimaksud Tree?
2. Apa fungsi root pada Tree?
3. Apa perbedaan parent dan child?
4. Apa yang dimaksud leaf node?
5. Mengapa Tree cocok untuk struktur hierarki?
6. Apa perbedaan preorder, inorder, dan postorder?
7. Apa perbedaan Linked List dan Binary Tree?

10. FORMAT LAPORAN

1. Judul Praktikum
2. Tujuan
3. Dasar Teori
4. Langkah Praktikum
5. Hasil dan Screenshot
6. Analisis
7. Kesimpulan